**Laporan Praktikum Algoritma & Pemrograman**
**Semester Genap 2025/2026**

**SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.**

**SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.**

| | |
|---|---|
| **NIM** | **71251233** |
| **Nama Lengkap** | **Mikael Gratianus Satrio Adi Kuncoro** |
| **Minggu ke / Materi** | **06 / Struktur Kontrol Perulangan** |

**PROGRAM STUDI INFORMATIKA**
**FAKULTAS TEKNOLOGI INFORMASI**
**UNIVERSITAS KRISTEN DUTA WACANA**
**YOGYAKARTA**
**2026**

## Definisi Perulangan

Program dapat diatur secara sekuensial, percabangan, perulangan, maupun kombinasi dari ketiganya. Pengaturan tersebut biasa disebut sebagai struktur kontrol. Perulangan digunakan apabila dalam program diperlukan untuk:

- Melakukan suatu hal yang sama beberapa kali.
- Melakukan suatu hal secara bertahap, di mana setiap tahap sebenarnya memiliki langkah yang sama.
- Mengakses sekumpulan data dalam suatu struktur data (List, Tuple, Queue, Stack dan beberapa struktur data lainnya).

Pada Python, perulangan dapat dilakukan dengan menggunakan **for**, **while** maupun dengan cara rekursif.

## Bentuk Perulangan for

Pada python, perulangan dapat dinyatakan dalam bentuk **for** dan **while**. Perlulangan for baisanya digunakan pada kondisi:

- **Jumlah perulangan sudah diketahui sejak awal,** Contohnya, membaca 10 file teks secara berurutan. Meskipun isi setiap file berbeda, proses membaca setiap file umumnya sama, sehingga bisa dilakukan secara berulang dari file pertama hingga file kesepuluh.
- **Perulangan terjadi karena operasi yang sama pada suatu rentang data atau rentang nilai.** Contohnya dalam mencari jumlah dari 100 bilangan pertama, maka secara berturut-turut dilakukan penjumlahan dari 1 sampai 100 (1+2+3…+100).

Perulangan for pada rentang tertentu lebih mudah dilakukan dengan menggunakan bantuan fungsi range(), yang bentuknya sebagai berikut:

- Range(stop). Digunakan untuk menghasilkan rentang dari 0 sampai stop-1. Misalkan range(6), berarti menghasilkan rentang 0-5, 6 tidak termasuk.
- range(start, stop, step). Digunakan untuk menghasilkan rentang dari start, sampai stop dengan peningkatan sejumlah step.

Berikut ini adalah contoh program untuk menampilkan bilangan dari 1 sampai 100 dengan menggunakan for dan range():

```
for i in range(1, 101):
    print(i)
```

Pada program tersebut, ada perulangan for dengan menggunakan range(), dimulai dari 1 (start) sampai 101 (stop-1) dengan langkah 1 (default step adalah 1). Kemudian variabel i digunakan sebagai counter, di mana nilai i akan naik secara berurutan sesuai dengan nilai yang dihasilkan dari fungsi range() tersebut. Untuk kasus di mana tidak diperlukan counter, maka bentuk perulangan seperti berikut ini:

```
for _ in range(1, 101):
     print('Hello World')
```

Program tersebut akan menampilkan tulisan Hello World sebanyak 100 kali, yang tidak membutuhkan nilai dari suatu counter.

**-(Step negatif)**

Perhatikan perulangan for berikut ini yang akan menampilkan seluruh bilangan genap dari 2 sampai 100:

```
for i in range(2, 101, 2):
    print(i)
```

Perulangan dilakukan pada rentang 2-100, dengan langkah 2. Maka rentang yang dipakai adalah 2,4, 6, 8, 10, 12, ..., 100. Jika diperlukan untuk menampilkan bilangan genap dari 100 sampai 2?, Maka bisa menggunakan fungsi renge() step negatif, seperti pada contoh berikut ini:

```
for i in range(100, 1, -2):
    print(i)
```

Program tersebut akan menampilkan bilangan genap mulai dari 100, 98, 96, 94, ..., sampai 2.

### Bentuk Perulangan While

Bentuk while biasanya digunakan pada kondisi di mana jumlah perulangan belum diketahui sebelumnya. Bentuk perulangan while secara umum adalah sebagai berikut:

```
<start>
while <stop>:
    operation
```

operation

<step (optional)>

Berikut ini adalah contoh perulangan dengan menggunakan while yang digunakan untuk memastikan input yang dimasukkan oleh pengguna adalah bilangan genap:

```
bilangan = 0
genap = False
while genap == False:
    bilangan = int(input('Masukkan bilangan genap: '))
    if bilangan % 2 == 0:
        genap = True
print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Output yang dihasilkan oleh program tersebut dapat dilihat pada Gambar 1.1. Pengguna awalnya memasukkan bilangan ganjil, tetapi program terus meminta pengguna memasukkan bilangan genap. Program berhenti setelah pengguna memasukkan 88, yang merupakan bilangan genap. Kasus ini sangat sesuai jika menggunakan perulangan while, karena tidak diketahui sampai berapa kali pengguna memasukkan bilangan ganjil yang tidak sesuai dengan permintaan.

```
C:\ProgramData\Microsoft\Win
(base) PS C:\Users\yuanl\Documents\Github\modul-alpro-python
Masukkan bilangan genap: 99
Masukkan bilangan genap: 23
Masukkan bilangan genap: 11
Masukkan bilangan genap: 37
Masukkan bilangan genap: 87651
Masukkan bilangan genap: 88
88 yang anda masukkan adalah bilangan genap
```

Gambar 1.1: Penggunaan while untuk mengambil input dari pengguna sampai sesuai dengan permintaan.

### Penggunaan Break dan Continue

Perulangan dapat dikontrol dengan menggunakan break dan continue. Break digunakan untuk menghentikan perulangan, sedangkan continue digunakan untuk melanjutkan perulangan ke iterasi berikutnya. Perhatikan program berikut ini yang akan menampilkan bilangan dari 1 sampai 10:

```
for i in range(1, 11):
    print(i)
print('Selesai')
```

Program tersebut akan menampilkan bilangan 1 sampai 10, kemudian pada baris terakhir muncul tulisan 'Selesai'. Jika diinginkan perulangan yang seharusnya sampai 10, dihentikan saat mencapai 5, maka diperlukan break dengan kondisi seperti program berikut ini:

```
for i in range(1, 11):
    if i == 5:
        break
    else:
        print(i)
print('Selesai')
```

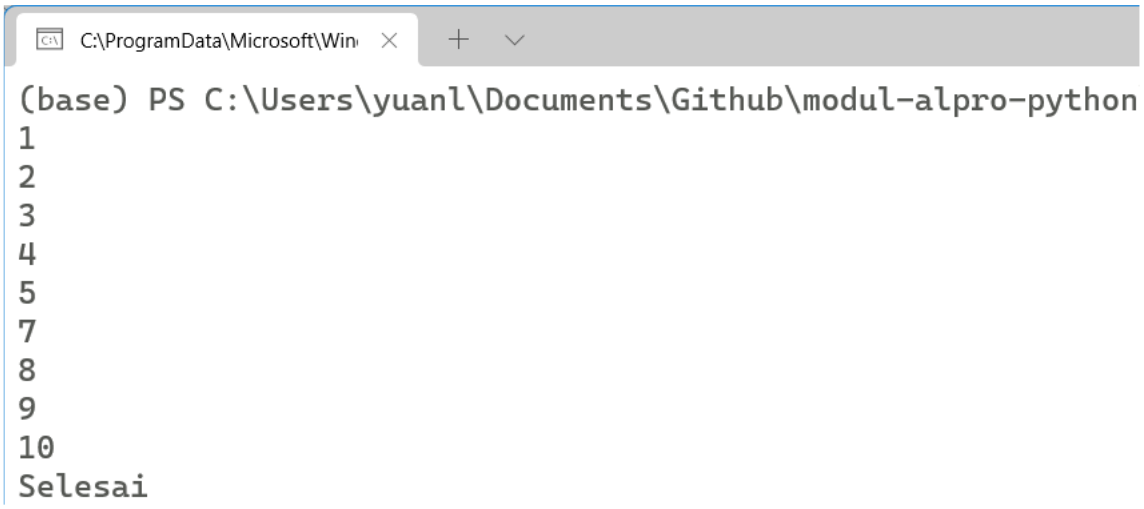
Pada saat nilai i=5, maka boolean expression dari i==5 akan menghasilkan nilai True, program akan menjalankan instruksi break. Maka selanjutnya perulangan akan dihentikan dan program berlanjut pada baris setelah perulangan tersebut, yaitu menampilkan tulisan 'Selesai'.

Continue digunakan untuk melewati langkah perulangan saat ini dan langsung melanjutkan ke langkah berikutnya. Misalnya, menampilkan angka 1 sampai 10 tapi melewatkan angka 6. Berikut contoh nya:

```
for i in range(1, 11):
    if i == 6:
      continue
    else:
```

```
    print(i)
print('Selesai')
```

Output yang dihasilkan dari contoh penggunaan continue dapat dilihat pada Gambar 1.2. Program menampilkan angka dari 1 sampai 10, tetapi melewatkan angka 6.




Gambar 1.2: Penggunaan continue untuk melewatkan iterasi saat counter(i) bernilai 6.

### Konversi dari Bentuk for Menjadi Bentuk while

Bentuk perulangan for sebagian besar dapat dikonversi menjadi bentuk while. Beberapa hal yang ada di bentuk for dan while adalah sebagai berikut:

- Harus ada nilai awal, untuk memulai perulangan. Nilai ini biasanya digunakan sebagai nilai awal variabel penghitung.
- Harus ada nilai akhir, untuk mengakhiri perulangan. Perulangan akan terus berjalan sampai kondisi batas ini tercapai.
- Harus ada langkah (increment atau decrement), agar nilai variabel berubah setiap iterasi sehingga perulangan dapat bergerak dari nilai awal menuju nilai akhir.
- Harus ada kondisi perulangan, yaitu syarat yang menentukan apakah perulangan masih dijalankan atau dihentikan.

Misalkan ada perulangan for sederhana seperti berikut:

```
for i in range(1, 11):
    print(i)
```

maka dapat diidentifikasi bahwa perulangan tersebut dimulai dari 1, berakhir di 10, dengan step adalah 1. Konversi ke bentuk while dapat dilakukan dengan mudah, perulangan while berikut ini akan menghasilkan output yang sama:

```
i = 1                   //awal
while i <= 10:          //kondisi akhir
    print(i)
    i = i + 1           //step
```

### LINK GITHUB

: [https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git](https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git)

### SOAL 1

```
soal01.py > ...
def perkalian(a, b):
    hasil = 0
    string = ""
    for i in range(a):
        hasil += b
        if i < a -1:
            string += str(b) + " + "
        else:
            string += str(b)
    print(str(a) + " x " + str(b) + " = " + string + " = " + str(hasil) + ".")
    return hasil

perkalian(6,5)
perkalian(7,10)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu06> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu06\soal01.py"
6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30.
7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70.
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu06>
```

Gambar 1.3: *Source code* soal latihan mandiri 01

Penjelasan:

Ini adalah program yang menerapkan perhitungan perkalian dengan menggunakan penjumlahan berulang melalui fungsi **perkalian(a, b)**. Parameter **a** menyatakan jumlah perulangan, sedangkan **b** adalah angka yang dijumlahkan. Variabel **hasil** digunakan untuk menyimpan total penjumlahan dengan nilai awal **0**, dan variabel **string** digunakan untuk menyimpan teks penjumlahan yang akan ditampilkan. Program kemudian melakukan perulangan **for i in range(a)** yang membuat urutan angka mulai dari 0 sampai sebelum a. Pada setiap perulangan, nilai b ditambahkan ke hasil. Jika perulangan belum terakhir **(i < a - 1)**, maka **b** ditambahkan ke string diikuti tanda **" + ",** sedangkan pada perulangan terakhir hanya

ditambahkan **b** tanpa tanda tambah. Setelah perulangan selesai, program mencetak hasil dalam bentuk penjumlahan dan hasil perkalian, lalu return hasil mengembalikan nilai hasil perkalian. Fungsi lalu dipanggil dengan perkalian(6,5) dan perkalian(7,10) untuk menampilkan hasilnya.

## SOAL 2

```
soal02.py > ...
def ganjil(batas_bawah, batas_atas):
    if batas_bawah < batas_atas:
        hasil = []
        for i in range(batas_bawah, batas_atas +1):
            if i % 2 != 0:
                hasil.append(i)
        print(f"bawah = {batas_bawah}, atas = {batas_atas}. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: ", end="")
        print(*hasil, sep=", ", end=".\n")

    else:
        hasil = []
        for i in range(batas_bawah, batas_atas - 1, -1):
            if i % 2 != 0:
                hasil.append(i)
        print(f"bawah = {batas_bawah}, atas = {batas_atas}. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: ", end="")
        print(*hasil, sep=", ", end=".\n")

batas_bawah = int(input("Masukkan batas bawah: "))
batas_atas = int(input("Masukkan batas atas: "))

ganjil(batas_bawah, batas_atas)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu06> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu06\soal02.py"
Masukkan batas bawah: 10
Masukkan batas atas: 30
bawah = 10, atas = 30. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29.
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu06> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu06\soal02.py"
Masukkan batas bawah: 97
Masukkan batas atas: 82
bawah = 97, atas = 82. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83.
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu06>
```

Gambar 1.4: *Source code* soal latihan mandiri 02

Penjelasan:

Ini adalah program yang dapat menampilkan deret bilangan ganjil dari batas bawah dan batas atas yang diberikan oleh pengguna. Pertama program diawali dengan fungsi bernama **ganjil(batas_bawah, batas_atas)** yang memiliki dua parameter yaitu **batas_bawah** dan **batas_atas** untuk menentukan rentang angka yang akan dicek. Program memeriksa apakah **batas_bawah lebih kecil dari batas_atas**. Jika benar, program membuat list kosong bernama **hasil** untuk menyimpan bilangan ganjil, lalu melakukan **perulangan dari batas_bawah sampai batas_atas** menggunakan **range(batas_bawah, batas_atas + 1) +1** itu agar batas atas tetap ikut dihitung. Pada setiap perulangan, program mengecek apakah angka tersebut **ganjil** dengan kondisi **i % 2 != 0**. Jika ganjil, angka tersebut dimasukkan ke dalam list **hasil**. Setelah perulangan selesai, program mencetak keterangan serta semua angka ganjil yang ditemukan. Jika kondisi awal tidak terpenuhi (**batas_bawah ≥ batas_atas**), maka program tetap membuat list **hasil**, tetapi perulangannya dilakukan **dari besar ke kecil** menggunakan **range(batas_bawah, batas_atas - 1, -1)**. Proses pengecekan bilangan ganjil

dan penyimpanan ke dalam list dilakukan dengan cara yang sama, lalu hasilnya dicetak. Di bagian akhir program, pengguna diminta memasukkan **batas bawah** dan **batas atas** menggunakan input() yang kemudian diubah menjadi bilangan bulat dengan **int(),** setelah itu fungsi **ganjil()** dipanggil untuk menampilkan deret bilangan ganjil sesuai rentang yang dimasukkan pengguna.

## SOAL 3

```
soal03.py > ...
print("Program penghitung IPS Mahasiswa")
jumlah_mk = int(input("Berapa jumlah mata kuliah? "))

total_bobot = 0
sks_per_mk = 3

for i in range(1, jumlah_mk + 1):
    nilai = input(f"Nilai MK {i}: ").upper()

    if nilai == "A":
        bobot = 4
    elif nilai == "B":
        bobot = 3
    elif nilai == "C":
        bobot = 2
    elif nilai == "D":
        bobot = 1
    else:
        bobot = 0

    total_bobot += bobot * sks_per_mk

total_sks = jumlah_mk * sks_per_mk
ips = total_bobot / total_sks

print(f"Nilai IPS anda semester ini {ips:.2f}")
```

```
Berapa jumlah mata kuliah? 6
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: C
Nilai MK 4: A
Nilai MK 5: D
Nilai MK 6: C
Nilai IPS anda semester ini 2.67
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu06>
```

Gambar 1.5: *Source code* soal latihan mandiri 03

Penjelasan:

Ini adalah program penghitung nilai Indeks Prestasi Semester (IPS). Program diawali dengan mencetak judul “Program penghitung IPS Mahasiswa” dengan menggunakan fungsi **print().** Setelah itu, program meminta pengguna memasukkan jumlah mata kuliah menggunakan **input()** lalu diubah menjadi bilangan bulat dengan **int()** yang disimpan dalam variabel **jumlah_mk**. Lalu lanjut ke variabel **total_bobot** dengan nilai awal 0 untuk menyimpan jumlah bobot nilai, serta **sks_per_mk = 3** yang berarti setiap mata kuliah memiliki 3 SKS. Selanjutnya program melakukan perulangan sebanyak jumlah mata kuliah menggunakan **for i in range(1, jumlah_mk + 1)** dan meminta pengguna memasukkan nilai huruf pada setiap mata kuliah, lalu mengubahnya menjadi huruf besar dengan sebuah method yaitu **upper()** yang disimpan pada variabel **nilai.** Nilai huruf tersebut kemudian dikonversi menjadi bobot angka **(A=4, B=3, C=2, D=1, selain itu=0)**, lalu bobot dikalikan

dengan SKS dan ditambahkan ke variabel **total_bobot**. Setelah semua nilai dimasukkan, lalu lanjut ke menghitung **total SKS** dengan mengalikan jumlah mata kuliah dengan SKS per mata kuliah, kemudian menghitung **IPS** dengan membagi total bobot dengan total SKS. Terakhir, program menampilkan **hasil IPS** dengan dua angka di belakang koma.